```
  'use server';
  const supportedModel = getSupportedModel(provider, model);
  const history = getMutableAIState();
  const formattedPrompt = await prompt.format({
    input
  });
  const result = await streamUI({
    model: supportedModel,
    messages: [...history.get(), { role: 'user',
content: formattedPrompt }],
    text: ({ content, done }) => {
      if (done) {
        history.done([...history.get(), { role: 'assistant', content }]);
      }
      return <ChatBubble role="assistant"
text={content} className={`mr-auto border-none`} />;
    },
  });

  return {
    id: generateId(),
    role: 'assistant',
    display: result.value,
  };
}
```

Formats the user input using the imported prompt template before sending it to the AI model

Incorporates the formatted prompt into the message history for streaming responses

The code now imports the exported prompt template from listing 6.2. Within the `continueConversation` function, user input is formatted using `prompt.format()`, generating a structured prompt that includes relevant examples. This formatted prompt is then included in the `streamUI` call as part of the message history as well.

Project: chat-rsc-langchain

The code for this example project is in the ch06/chat-rsc-langchain folder. You can use that example by running this command in the root folder:

```
$ npm run dev -w ch06/chat-rsc-langchain
```

You'll need an OpenAI API key to access their REST API or a Google API key for Google services. For details on setting up and creating API keys, refer to the appendix.

```
// .env

OPENAI_API_KEY=<SECRET_KEY>
GEMINI_API_KEY=<YOUR_GEMINI_API_KEY>
```

WARNING Make sure that your Google/Open API key is valid to avoid any problems during the installation process. Remember, this secret key should be kept confidential and not shared publicly.